

SOMMAIRE

- | | |
|-------------------|------------------|
| 1. Groupe | • Arduino |
| 2. Idée | • Dashboard & IA |
| 3. Objectif | 5. Organisation |
| 4. Fonctionnement | 6. Synthèse |
| • Base de données | |

1. GROUPE

Équipe Projet GRAVITY - Workshop 2026 :

Tasnim Alioui

Rémi Pernak

Imran Lamkadem

Thomas Lesage

2. IDÉE

Une mission spatiale longue durée pose un problème médical concret : en microgravité, le corps humain se dégrade de façon lente et mesurable (perte de densité osseuse, fonte musculaire, baisse des capacités cardiovasculaires).

Sur Terre, une équipe médicale surveillerait ces constantes en temps réel. Mais en raison des délais de communication spatiale de plusieurs minutes, la surveillance doit vivre à bord, sans dépendre d'Internet ni d'un lien permanent.

GRAVITY est né de ce constat : créer un système de santé connecté embarqué et autonome fonctionnant selon le principe « **offline-first** ».

3. OBJECTIF

GRAVITY remplit trois missions principales :

1. **Suivre l'évolution physiologique** de chaque membre d'équipage par rapport à son état de référence mesuré avant le départ (sa « baseline »).
2. **Détecter les dérives** avant qu'elles ne deviennent des problèmes critiques, grâce à une analyse à trois niveaux (écart instantané, tendance sur un mois et score de santé composite).
3. **Proposer des contre-mesures physiques adaptées** via la génération de plans d'entraînement correctifs et répondre aux questions de l'équipage grâce à une IA conversationnelle locale.

4. FONCTIONNEMENT

Le système fonctionne en boucle continuus : le collecteur (ou simulateur) transmet les mesures au cœur du système (GRAVITY Core sur FastAPI), qui les valide et les enregistre.

À chaque mesure, l'état de santé est réévalué. Le Dashboard interroge en permanence ce cœur pour afficher l'état de l'équipage, tandis que l'IA consulte le système via « Tool Calling » pour répondre de façon fiable aux questions.

Modèle Conceptuel de Données (MCD)

Entité	Description	Attributs principaux
Membre de l'équipage	Un astronaute suivi par le système.	identifiant , nom (Crew01...), rôle
Baseline	État physiologique de référence mesuré avant le départ.	indicateur , valeur de référence , date d'enregistrement
Mesure	Valeur relevée sur un indicateur à une date donnée.	indicateur , valeur , date de mission , date de réception
Seuil d'alerte	Paramètres de détection d'écart et de tendance.	indicateur , écart alerte/critique , pente max tolérée
Événement	Fait de mission (blessure, fatigue, etc.).	type , détails , date de mission
Alerte	État courant d'un indicateur dégradé mis à jour en continu.	indicateur , niveau , message , date de création

Le modèle gère le **double horodatage** (date de mission simulée et date réelle de réception), permettant d'accélérer le temps sans corrompre les calculs de tendance.

Arduino

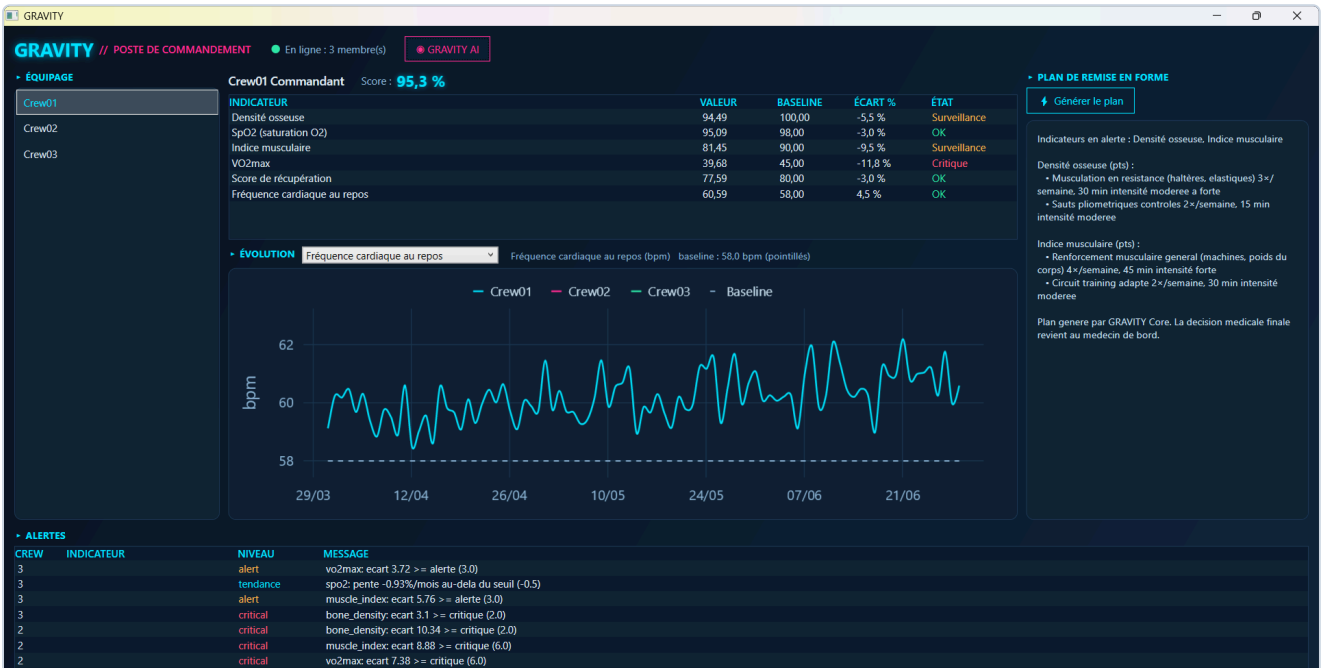
Réalisé par : [Tasnim Alioui & Rémi Pernak](#)

Dans la version réelle du système, un module **Arduino** (ou Raspberry Pi) fait office de collecteur physique. Il capture les métriques physiologiques depuis des capteurs embarqués et les transmet au GRAVITY Core sous forme de structures JSON standardisées. Un simulateur logiciel en Python permet également de générer ces flux pour tester les scénarios de dérive sur plusieurs mois.

Dashboard & IA

Réalisé par : [Imran Lamkadem & Thomas Lesage](#)

Le **Dashboard (Poste de Commandement)** est développé en C# WPF. Il offre une vue complète sur l'état de santé de l'équipage, les alertes en temps réel et les plans de remise en forme générés.



- **Plans de remise en forme adaptatifs** : Génération automatique d'exercices correctifs ajustés selon les blessures (ex. musculation en résistance, sauts pliométriques).
- **Assistant IA intégré (GRAVITY AI)** : Le dashboard dispose d'un bouton dédié **GRAVITY AI** permettant d'ouvrir une fenêtre de chat conversationnel. Les membres de l'équipage peuvent poser leurs questions directement en langage naturel (ex. « comment va Crew02 ? »). L'IA (modèle local Qwen 14B via Ollama) interroge le cœur via *Tool Calling* pour récupérer les données réelles et fournir une réponse précise sans jamais altérer ni halluciner les données de la base.

5. ORGANISATION

Réalisé par : Rémi Pernak

Pour assurer un suivi rigoureux et une bonne coordination de l'équipe, nous avons mis en place :

- **Gestion de projet GitHub** : Création d'un dépôt GitHub centralisant l'ensemble du code source, ainsi que d'un projet GitHub (Kanban / Project Board) pour lister, attribuer et planifier les différentes tâches techniques au sein de l'équipe.

Le développement s'est structuré selon le phasage suivant :

1. **Modélisation & BDD** : Conception du MCD Merise et déploiement de PostgreSQL.
2. **Cœur & Moteur de règles (GRAVITY Core)** : Développement de l'API FastAPI pour traiter les mesures, déceler les dérives (écarts instantanés, pentes sur 30 jours) et calculer le score composite.
3. **Collecte (Arduino / Simulateur)** : Développement du script de simulation et intégration des flux de données JSON.
4. **Interface utilisateur (Dashboard WPF)** : Création du poste de commandement graphique C# et intégration des graphiques d'évolution.
5. **Intégration de l'IA locale (GRAVITY AI)** : Configuration de Qwen via Ollama, intégration des fonctions de Tool Calling et sécurisation des communications via Tailscale.

6. SYNTHÈSE

GRAVITY offre une solution de suivi médical autonome et hautement résiliente, parfaitement adaptée aux contraintes des vols spatiaux habités.

En combinant un moteur de règles déterministe pour la détection préventive des dérives physiologiques et une IA conversationnelle locale embarquée dans le dashboard, le système garantit un suivi médical fiable, sécurisé et 100 % opérationnel hors ligne.